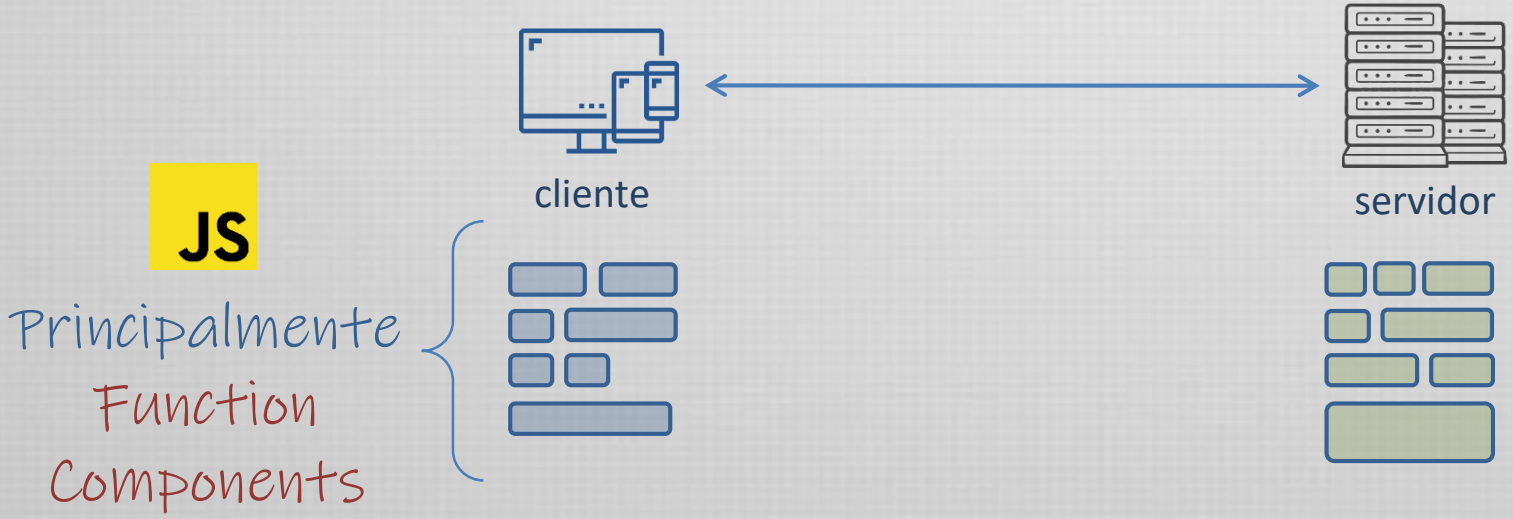
Ingeniería de Aplicaciones Web

Diego C. Martínez

Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur

Modularizar es necesario

Las Aplicaciones Web son sistemas complejos



Modularizar es necesario

Las Aplicaciones Web son sistemas complejos

cliente

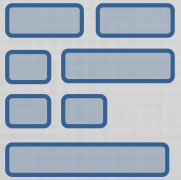
servidor

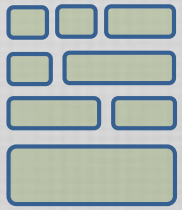
JS

Principalmente

Function

Components





Clases OOP

```
public class Users {
…
}
```

Modularizar es necesario

Las Aplicaciones Web son sistemas complejos

cliente

servidor

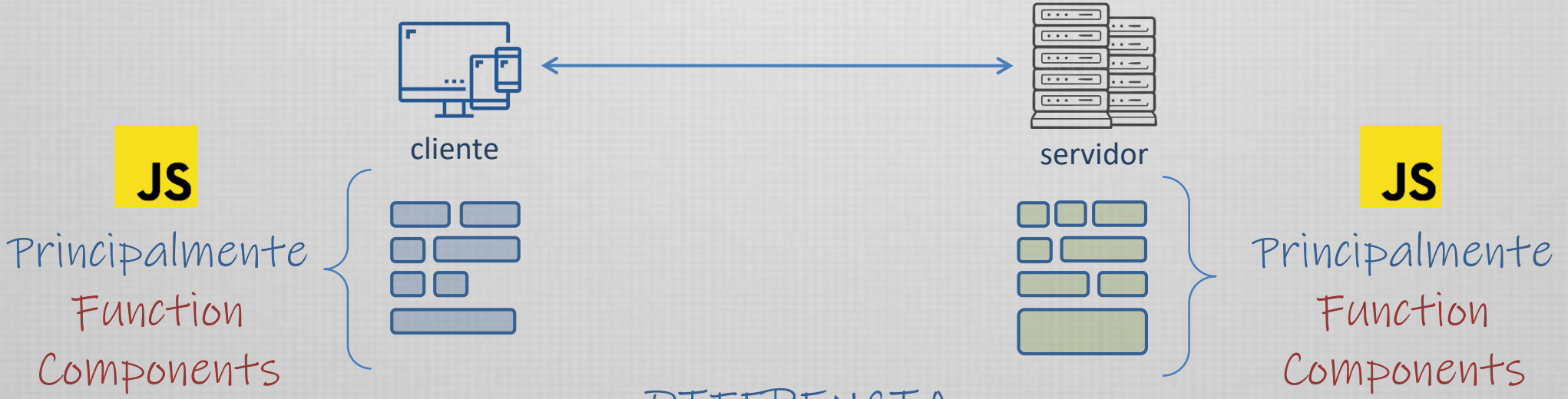
JS

Principalmente
Function
Components

Clases o
Módulos con
Procedimientos

Modularizar es necesario

Las Aplicaciones Web son sistemas complejos



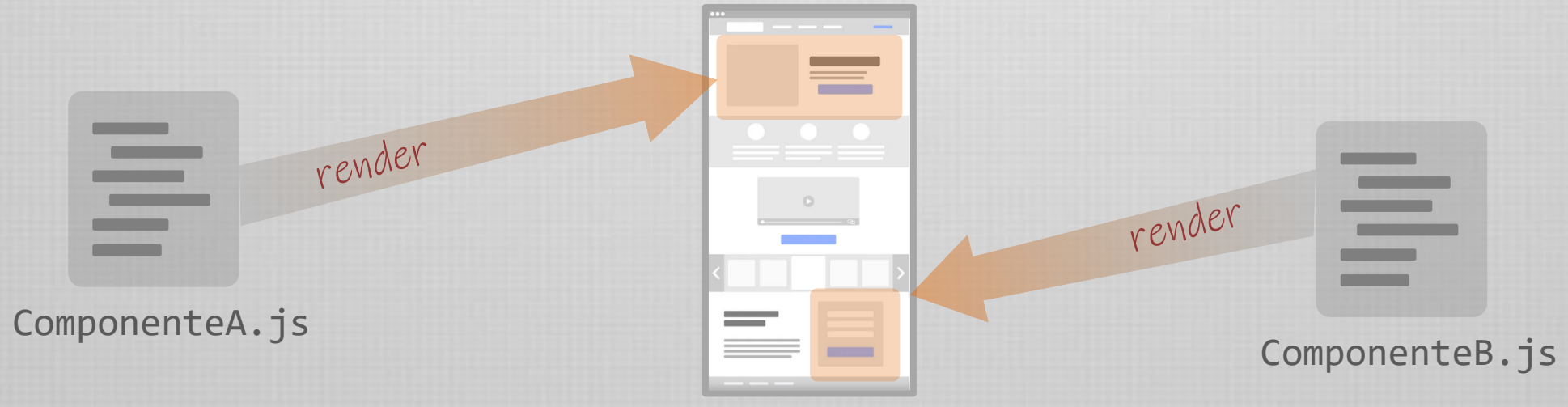
DIFERENCIA

En el frontend, los componentes modelan la interfaz

En el backend, los componentes modelan lógica

Comparten la forma, pero no la responsabilidad.

Function Components



Function Components

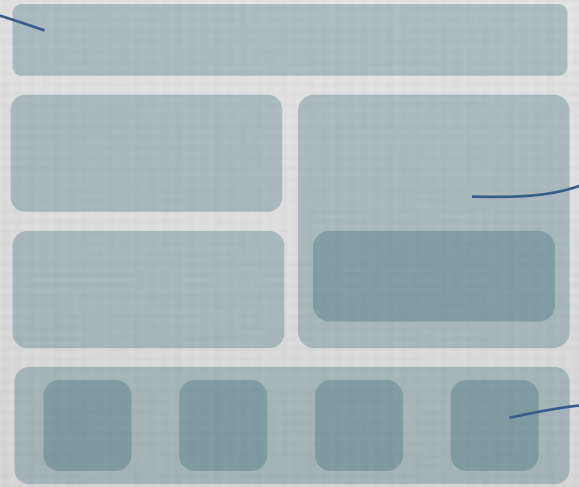
```
function MiComponente() {

    // datos

    const funcionX = () =>{
        // función auxiliar
    }

    return (
        // renderizamos en JSX
        <div>…</div>
     );
}

export default MiComponente;
```

MiComponente.js

Esto es un fragmento de la UI

Function Components

```
function CompA() {
    return ( <div>…</div>);
}
export default CompA;
```



```
function CompB() {
    return ( <div>…</div>);
}
export default CompB;
```

```
function CompC() {
    return ( <div>…</div>);
}
export default CompC;
```

Function Components

```
function CompB() {
    return ( <div>…</div>);
}
export default CompB;
```

```
function CompA() {
    return ( <div>…</div>);
}
export default CompA;
```

```
function CompC() {
    return ( <div>…</div>);
}
export default CompC;
```

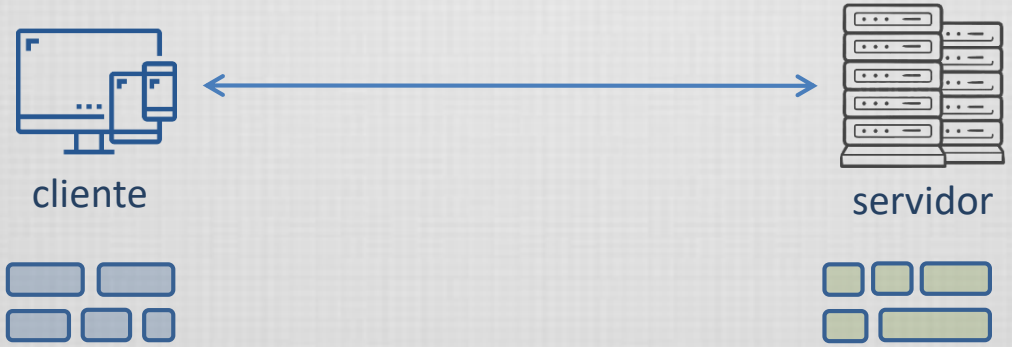
Function Components

```
function CompA() {
    return ( <div>…</div>);
}
export default CompA;
```

```
function CompC() {
    return ( <div>…</div>);
}
export default CompC;
```

```
function CompB() {
    return ( <div>…</div>);
}
export default CompB;
```

Escenario web



Client Components

Si el código necesita interactuar con el usuario (estado, clicks, efectos) va del lado del cliente

Server Components

Si el código necesita acceder a la base de datos, secretos o simplemente renderizar contenido, mejor que quede en el servidor

```
'use client'
```

Puede usar "cosas" del navegador

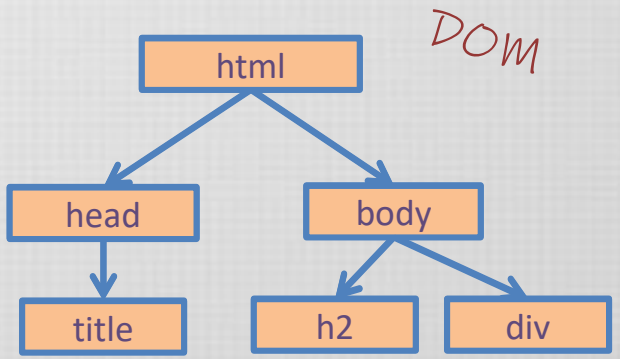
(window, document, localStorage…)

Puede usar "cosas" de UI

(onClick, onChange, etc)

React – Virtual DOM (solo cliente)

```
<html>
 <head>
      <title>Hola</title>
 </head>
 <body>
      <h2>Blog</h2>
      <div>Bla bla bla</div>
 </body>
</html>
```



Re-rendering

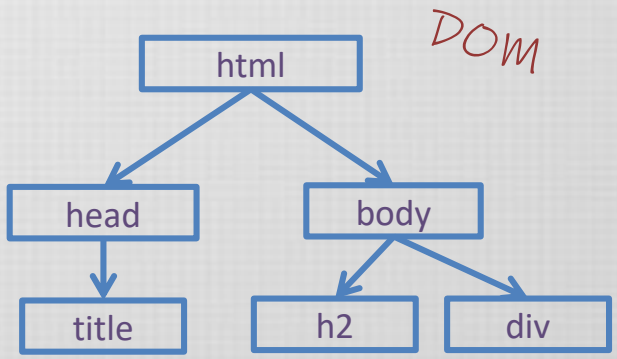
Muy Procedural

1. onclick()
    1. getElementByID
    2. Modificar datos
    3. Append
    4. getElementByID
2. ...

JS

React – Virtual DOM (solo cliente)

```
<html>
 <head>
      <title>Hola</title>
 </head>
 <body>
      <h2>Blog</h2>
      <div>Bla bla bla</div>
 </body>
</html>
```

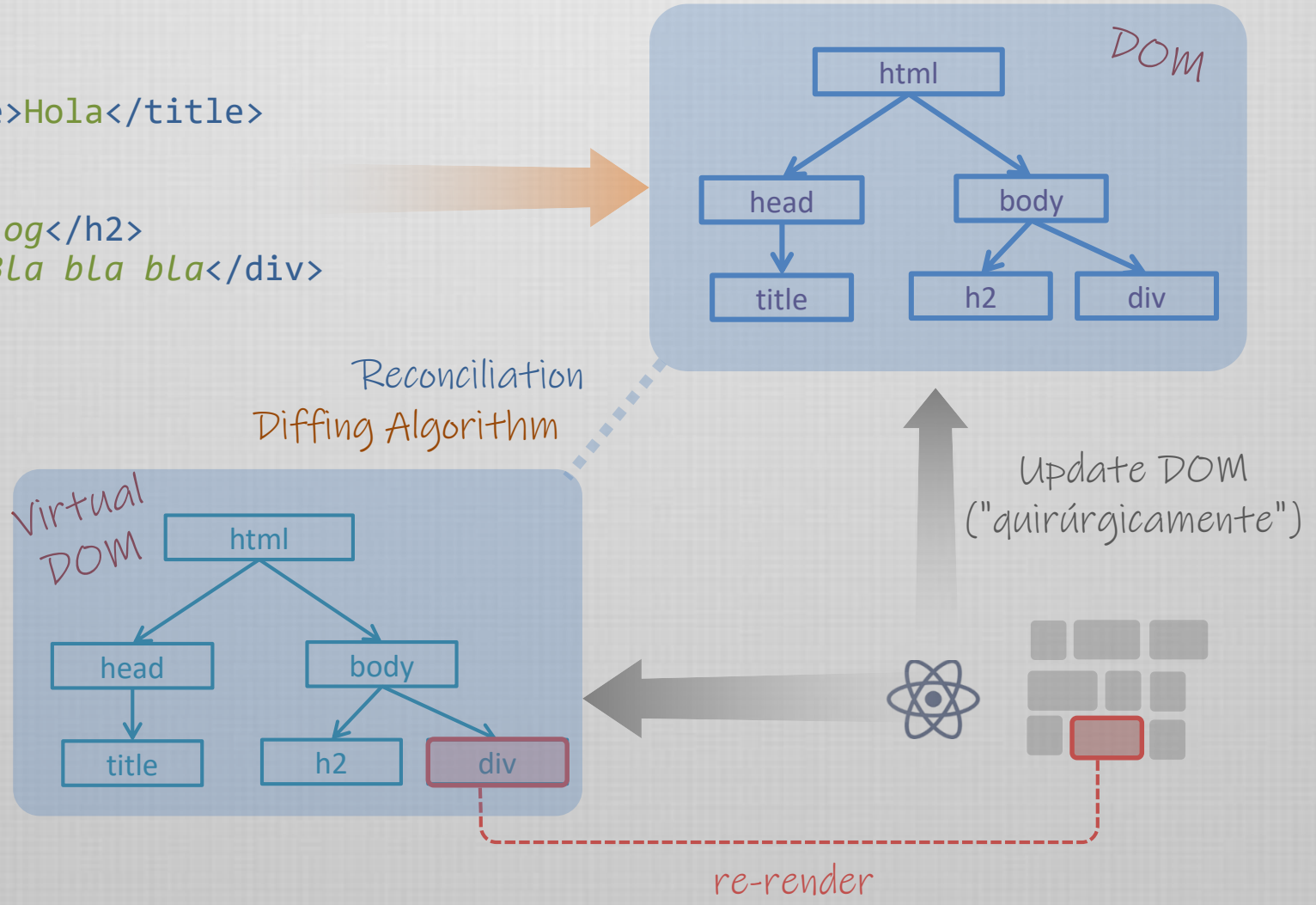


Declarativo es mejor

```
<Banner>
<TodoList orden="desc">
<ConfirmacionCompra>
```

React – Virtual DOM (solo cliente)

```
<html>
 <head>
      <title>Hola</title>
 </head>
 <body>
      <h2>Blog</h2>
      <div>Bla bla bla</div>
 </body>
</html>
```



Hooks

Funciones de React que tratan aspectos a considerar durante el tiempo de vida de los componentes

(se enganchan en el ciclo de vida y su estado e intervienen, aunque no importa mucho el detalle)

Comienzan todas con el prefijo "use"

`useEffect, useState, useRef, useContext, useReducer` …

Se agregaron desde la versión 16.8 para simplificar varias operatorias recurrentes

useState

Probablemente el más común

Permite mantener datos y administrar el estado del componente

```
const [edad, setEdad] = useState(23)
```

valor inicial

variable

Función

Este componente se ejecuta en el navegador

```
'use client';

import { useState } from 'react';

export default function Counter() {

  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Contador: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Incrementar
      </button>
    </div>
  );
}
```

count inicia con 0
setCount actualiza el valor

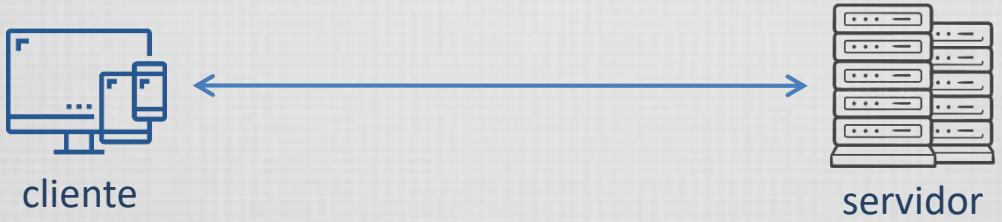
Incrementamos al hacer click
¡solo tiene sentido en el navegador!

```
'use client';

import { useState } from 'react';

export default function Counter() {

  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Contador: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Incrementar
      </button>
    </div>
  );
}
```

¿Cómo podemos hacer una página similar pero procesando en el servidor?

NEXT.JS

Next



Muestra la página
con el contador

Ofrece un link o botón para
pedirle al servidor
incrementar el contador

Debe llevar
registro del contador

Debe actualizar lo que ve el usuario
(devolver una página nueva)

Escenario web

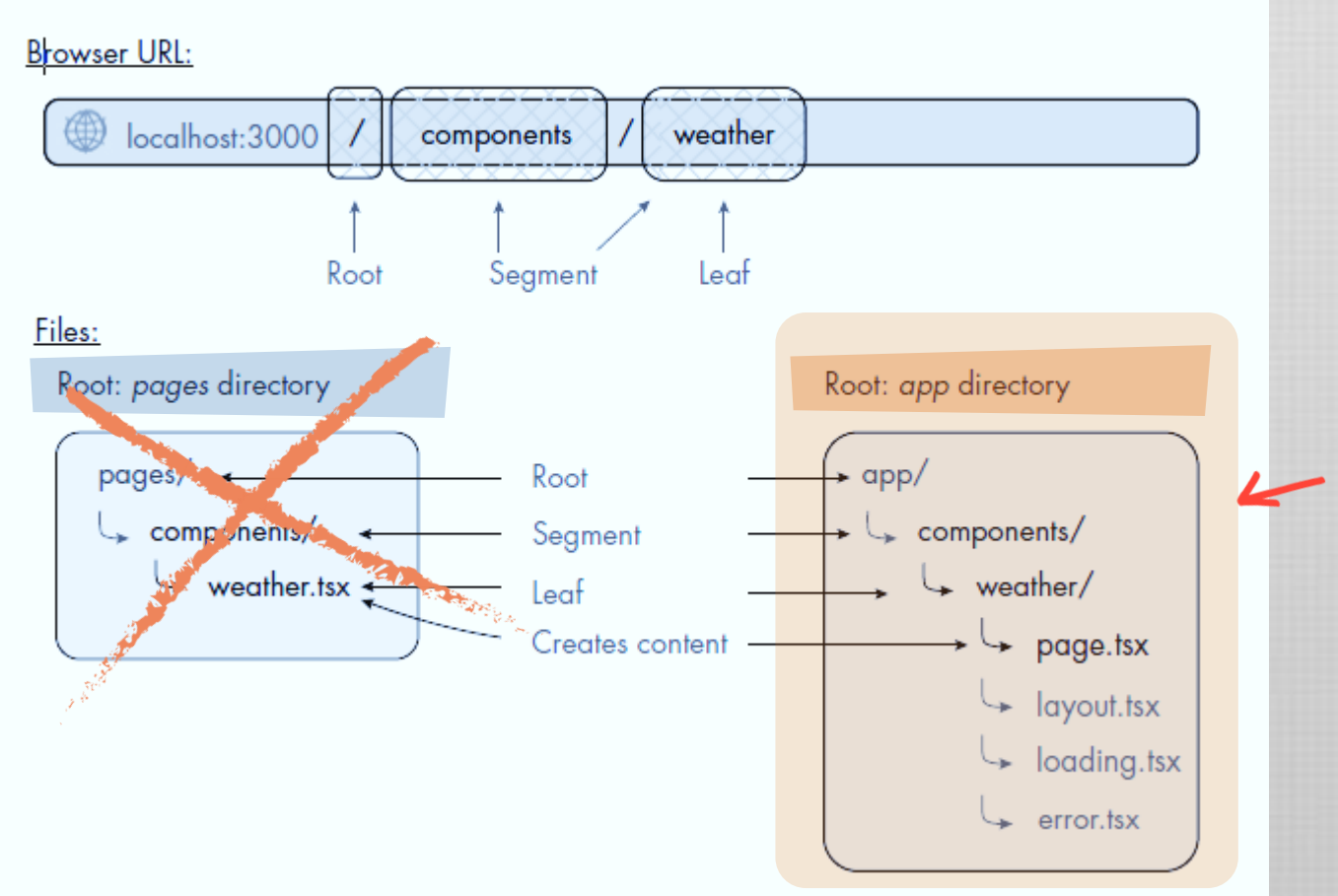
La comunicación hacia la aplicación es básicamente Mensaje HTTP + URL

La aplicación debe responder a cada pedido

GET + URL A	---------->	Realizar acción X
GET + URL B	---------->	Realizar acción Y
POST + URL C	---------->	Realizar acción Z

Todos los frameworks web nos ayudan a especificar esto

Rutas Next.JS



```
import { revalidatePath } from 'next/cache';

let count = 0;

async function increment() {
  'use server';
  count++;
  revalidatePath('/');
}

export default function Counter() {
  return (
    <div>
      <p>Contador: {count}</p>
      <form action={increment}>
        <button type="submit">Incrementar</button>
      </form>
    </div>
  );
}
```

Esta función se ejecuta
en el servidor
... pero la puede invocar desde el cliente

"Server Action"

NextJS agrega
toda la lógica necesaria
para que esto sea
un pedido al servidor
(HTTP GET, POST, etc)

Versión sin Server Action

```
export default function Counter({ searchParams }) {

  const count = Number(searchParams?.count ?? 0);

  return (
    <div>
      <p>Contador: {count} </p>

      <a href={`?count=${count + 1}`}>
        <button>Incrementar</button>
      </a>
    </div>
  );
}
```

useEffect

Permite implementar efectos "secundarios" luego del renderizado
Usualmente llamadas a APIs o actualizaciones del DOM

Accion a realizar

Bajo qué cambios

```
useEffect( función , [dependencia])
```